Sanjeevani Scan AI — Production, Security & Deployment Master Guide

> **Status:** Ready for Testing & Production Staging
> **Last Verified:** October 2026
> **Target Release:** Signed Android Standalone APK (`.apk` / `.aab`) with Hardened Anti-Reverse Engineering

1. System Architecture Overview

The application is structured according to the **Zero-Trust Client–Brain Architecture**, mirroring modern enterprise mobile AI applications (e.g., Google Lens, PlantNet):



2. Security & Anti-Reverse Engineering Protections

Your proprietary code, trained weights, and classical database are safeguarded through 5 active layers:

Layer	Mechanism	Protection Offered
Layer 1	**Hermes Bytecode (`.hbc`)**	React Native code is pre-compiled into binary machine bytecode. **No human-readable `.js` files exist in the release APK.** Decompilers (like JADX) only see binary bytecode.
Layer 2	**R8 / ProGuard Obfuscation**	Replaces all function, variable, and class names with scrambled symbols (e.g., `a.b()`). Strips debug symbols and metadata.
Layer 3	**Air-Gapped IP Isolation**	The neural network (`best_model_final.pth`), training datasets, and SQLite DB remain strictly on the private server. **They are never bundled into the user's phone.**
Layer 4	**Security Gateway Headers**	The API automatically injects `nosniff` , `X-Frame-Options: DENY` , `X-XSS-Protection` , and masks server software signatures.
Layer 5	**SSL Pinning & Integrity**	Prevents Man-In-The-Middle (MITM) packet sniffing via Burp Suite or Charles Proxy on production builds.

---

3. Verified Features Currently Active

1. **Leaf Identification**:
2. High-confidence classification for 66 plant classes.
3. Out-of-Distribution (OOD) rejection for non-leaf or unknown objects.
4. Grad-CAM AI heatmaps showing leaf venation patterns.
5. **Side-by-Side Visual Botanical Verification**:
6. Displays user-taken leaf side-by-side with official reference botanical photography.
7. Direct match percentage badge and venation/apex matching tips.
8. **Ergonomic Scan Action**:
9. Vibrant "Scan Another Leaf" button positioned directly below the taken photo box.
10. Quick gallery and live camera shortcut buttons.
11. **Standalone Medical Disclaimer**:
12. Clean, distinct amber warning card positioned at the bottom of the scan view.
13. **Full Dynamic 6-Language Localization**:
14. Supports: **English (`en`)**, **Hindi (`hi`)**, **Kannada (`kn`)**, **Tamil (`ta`)**, **Telugu (`te`)**, and **Malayalam (`ml`)**.
15. Translates UI buttons, Glossary terms & definitions, Botanical taxonomy, and dynamic Ayurvedic profiles (**Dosha, Rasa, Guna, Virya, Vipaka, Prabhava**).
16. **Ayurvedic Flora Library**:
17. Localized common names (e.g., *गिलोय, नीम, बेल, घृतकुमारी*).
18. "View Full Profile" button opens an in-app encyclopedia modal.
19. **Semantic Symptom Search**:

20. Matches clinical symptoms (e.g. *fever*, *anxiety*, *skin disorders*) directly to classical Ayurvedic remedies.

---

4. Local Run & Verification Commands

Whenever you test the application on your computer and phone:

A. Start the Backend API Server

```
# From project root (d:\Sanjeevani Scan AI)
.venv\Scripts\python -m uvicorn src.api.main:app --host 0.0.0.0 --port 8000 --reload
```

B. Start the Expo Mobile Development Server

```
# In src/mobile directory
cd src/mobile
npx expo start
```

C. Run the Complete Deep-Run Test Suite

```
# Validates API, DB, Giloy/Aloe profiles, Predict, and Explainability
.venv\Scripts\python test_deep_run.py
```

---

5. Production Build Runbook (When Ready to Finalize)

When you are done testing and ready to package the final production app, follow these steps:

Step 1: Deploy the Backend (Zero Cost Options)

Deploy the FastAPI backend container to **Render**, **Hugging Face Spaces (Docker)**, or **Google Cloud Run**:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "src.api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Step 2: Configure Production URL in `App.js`

In `src/mobile/App.js`:

```
// Replace local IP with your live cloud backend
const API_BASE_URL = 'https://your-sanjeevani-api.onrender.com';
```

Step 3: Install & Configure EAS Build

```
cd src/mobile
npm install -g eas-cli
npx eas login
npx eas build:configure
```

Step 4: Build the Standalone Signed APK

```
# Builds a standalone APK ready to install directly on Android phones
npx eas build -p android --profile preview
```

EAS will output a direct download link to `Sanjeevani-Scan-AI.apk`.

---

6. Project Directory Map

```
D:\Sanjeevani Scan AI\
├── data/
│   └── processed/
│       ├── sanjeevani_kb.db          # SQLite Database (Plants, Profiles, Uses)
│       ├── ayurvedic_knowledge.json  # Multi-lingual JSON Knowledge Base
│       └── best_model_final.pth      # Trained PyTorch Model Weights
├── src/
│   ├── api/
│   │   ├── main.py                   # FastAPI REST API & Security Gateway
│   │   ├── explainability.py         # Grad-CAM Heatmap Logic
│   │   └── semantic_search.py        # Sentence-Transformer FAISS Engine
│   ├── db/
│   │   ├── schema.sql                # Relational Database Schema
│   │   ├── extract_imppat.py         # Enriched Plant Data Generator
│   │   └── populate_db.py            # SQLite Ingestion Script
│   ├── mobile/
│   │   ├── App.js                    # Complete React Native Mobile Application
│   │   ├── app.json                  # Expo Config (Hermes Bytecode Enabled)
│   │   └── package.json              # Mobile Dependencies
│   └── models/
│       ├── model.py                  # DenseNet-121 Architecture
│       └── export_onnx.py            # ONNX Conversion Script
├── test_deep_run.py                  # Automated End-to-End Test Suite
└── PRODUCTION_DEPLOYMENT_GUIDE.md    # This Master Document
```